

Aula 15: Aprendizagem Profunda em Tempo Contínuo

Neural ODEs e Physics-Informed Neural Networks (PINNs)

Eliezer de Souza da Silva

Mestrado e Doutorado em Ciência da Computação (MDCC)
Universidade Federal do Ceará

Novembro de 2025

Roteiro

- 1 Motivação: De Camadas Discretas para Dinâmicas Contínuas
- 2 Neural ODE: Forward Pass e Solução Numérica
- 3 Método Adjunto: A Solução Elegante
- 4 PINNs: Deep Learning que Satisfaz Leis Físicas
- 5 GNNs como PINNs e como Solvers Variacionais
- 6 Encerramento

Ligação Conceitual: ResNets $\Rightarrow$ ODEs I

Objetivo: entender por que redes profundas podem ser modeladas como sistemas dinâmicos contínuos.

Atualização ResNet (discreta)

$$h_{t+1} = h_t + f(h_t, \theta_t)$$

Ligação Conceitual: ResNets $\Rightarrow$ ODEs II

Interpretação como Método de Euler (Explícito)

$$\frac{h_{t+1} - h_t}{\Delta t} = f(h_t, \theta_t), \quad \Delta t = 1$$

Ligação Conceitual: ResNets $\Rightarrow$ ODEs III

Conexão com Sistemas Dinâmicos Discretos

- **RNNs/LSTMs:** O estado h_t evolui no tempo discreto com base numa função de transição f não-linear.

$$h_{t+1} = \text{GRU}(h_t, x_t)$$

- **GNNs:** O "Message Passing" (Aula 14) é uma atualização residual discreta no *espaço do grafo*, sendo a profundidade (k) o tempo de difusão.

$$h_i^{(k+1)} = h_i^{(k)} + \text{AGGREGATE}(\dots)$$

Ligação Conceitual: ResNets $\Rightarrow$ ODEs IV

Limite contínuo: Neural ODE

Se tomarmos $\Delta t \rightarrow 0$:

$$\frac{dh(t)}{dt} = f(h(t), t, \theta)$$

Ligação Conceitual: ResNets $\Rightarrow$ ODEs V

Demonstração Formal do Limite (Sketch)

Considere uma sequência de estados:

$$h_{k+1} = h_k + \Delta t f(h_k)$$

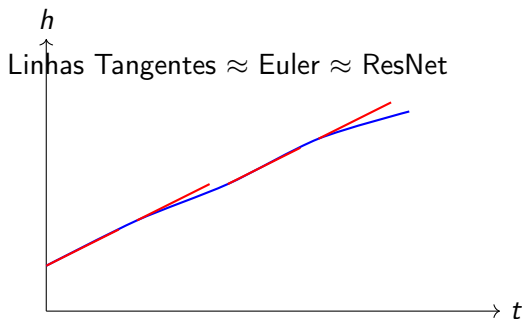
Defina $h(t_k) = h_k$ com $t_k = k\Delta t$. Então:

$$\frac{h_{k+1} - h_k}{\Delta t} = f(h_k)$$

$$\lim_{\Delta t \rightarrow 0} \frac{h(t + \Delta t) - h(t)}{\Delta t} = f(h(t))$$

$$\Rightarrow \frac{dh}{dt} = f(h(t))$$

Ligação Conceitual: ResNets $\Rightarrow$ ODEs VI



Solução Numérica via ODESolve I

Forward Pass

$$h(t_1) = h(t_0) + \int_{t_0}^{t_1} f(h(t), t, \theta) dt$$

ODE Solvers (Runge–Kutta, Dormand–Prince):

- passos adaptativos (mais passos onde a solução muda rápido);
- garantem erro local controlado;
- tornam o custo variável — uma rede “mais difícil” usa mais passos.

Solução Numérica via ODESolve II

Implicação Importante

O número de passos da rede **NÃO** é fixo. A profundidade efetiva muda durante o treino ou inferência.

Solução Numérica via ODESolve III

Solvers e Precisão

O "ODESolve" utiliza métodos numéricos.

- **Métodos de Passo Fixo (RK4):** Precisão garantida por pequeno Δt constante. Bom para implementações simples.
- **Métodos Adaptativos (Dopri5):**
 - **Funcionamento:** Ajustam Δt para garantir que o erro local está abaixo de um limiar.
 - **Vantagem:** Maior precisão com o menor número de passos possível.

Por que Backprop Não Funciona Direto?

Desenrolando o Solver no Backprop

Se o solver usa L passos:

$$\text{Memória} \propto L$$

Problema: L é variável e pode ser muito grande

Treinar redes profundas é difícil, mas treinar redes com profundidade dinâmica é pior:

- explosão de memória,
- instabilidade numérica,
- custo imprevisível.

O Dilema do Gradiente $\frac{\partial \mathcal{L}}{\partial \theta}$ I

Para treinar a rede $f(\cdot, \theta)$, precisamos de calcular o gradiente através de toda a trajetória do solver.

Autodiff Reverso Padrão (Gravação)

Custo de Memória: $O(L \cdot D)$

- Armazena todas as L ativações intermediárias $h(t_k)$.
- **Falha:** Se L é adaptativo e grande, a VRAM explode.

O Dilema do Gradiente $\frac{\partial \mathcal{L}}{\partial \theta}$ II

Autodiff Forward-Mode (Propagação)

Custo Computacional: $O(P \times L)$

- Propaga o estado e os P gradientes ($O(P \times D)$).
- **Falha:** Se P (parâmetros de f) é grande, o cálculo é muito lento.

A Solução Adjunta (Optimal)

O Método Adjunto ($O(1)$ em memória) é a única solução que se ajusta a ambas as restrições:

- $O(1)$ Memória (Constante).
- $O(L)$ Tempo (Apenas $2 \times$ Forward Pass).

A Solução: O Método Adjunto I

O Estado Adjunto ($a(t)$)

Define o gradiente da perda $\mathcal{L}$ em relação ao estado oculto h no tempo t :

$$a(t) \equiv \frac{\partial \mathcal{L}}{\partial h(t)}$$

A Solução: O Método Adjunto II

O Algoritmo: Duas ODEs

O gradiente é obtido resolvendo duas ODEs:

- 1 **Forward Pass:** Calcula $h(t_1)$.
- 2 **Backward Pass (ODE Adjunta):** Resolve o sistema para trás:

$$\frac{da(t)}{dt} = -a(t)^T \frac{\partial f}{\partial h}$$

Introduzindo o Vetor Adjunto I

Definição

$$a(t) = \frac{\partial \mathcal{L}}{\partial h(t)}$$

Introduzindo o Vetor Adjunto II

Queremos:

$$\frac{d}{dt}a(t)$$

Começamos da regra da cadeia:

$$a(t) = a(t + \Delta t) \frac{\partial h(t + \Delta t)}{\partial h(t)}$$

Usando Euler:

$$h(t + \Delta t) = h(t) + \Delta t f(h(t), t)$$

Então:

$$\frac{\partial h(t + \Delta t)}{\partial h(t)} = I + \Delta t \frac{\partial f}{\partial h}$$

Introduzindo o Vetor Adjunto III

Expandindo:

$$a(t + \Delta t) = a(t) + \Delta t a(t) \frac{\partial f}{\partial h}$$

Passando ao limite:

$$\frac{da}{dt} = -a(t)^\top \frac{\partial f}{\partial h}$$

Introduzindo o Vetor Adjunto IV

Resumo Final

Backward de Neural ODE:

$$\frac{da}{dt} = -a^\top \frac{\partial f}{\partial h}$$

Gradiente dos parâmetros:

$$\frac{d\mathcal{L}}{d\theta} = - \int_{t_1}^{t_0} a(t)^\top \frac{\partial f}{\partial \theta} dt$$

Métodos Avançados I

Trade-offs Avançados e Soluções Híbridas

O Método Adjunto é lento (custo de tempo alto) porque precisa de **re-calcular a trajetória** $h(t)$ durante o Backward Pass.

- **Métodos de Checkpointing:** Reduzem o custo de re-cálculo armazenando $h(t_k)$ em pontos discretos (checkpoints).
- **Trade-off:** Custo de Memória $O(\sqrt{L})$ (médio) e Custo de Tempo $O(L)$ (melhor que re-cálculo total).
- **Solvers Híbridos:** Usar solucionadores que são naturalmente mais rápidos ou mais estáveis, mesmo com o custo de tempo do Método Adjunto.

Exemplo 1: ODE Linear I

$$\frac{dh}{dt} = \theta h$$

Solução:

$$h(t) = h_0 e^{\theta t}$$

Loss:

$$\mathcal{L} = \frac{1}{2}(h(T) - y)^2$$

Exemplo 1: ODE Linear II

Cálculo do Adjunto

$$a(T) = h(T) - y$$

$$\frac{da}{dt} = -a(t)\theta \Rightarrow a(t) = a(T)e^{-\theta(T-t)}$$

Exemplo 1: ODE Linear III

Gradiente Final

$$\frac{\partial \mathcal{L}}{\partial \theta} = - \int_T^0 a(t) h(t) dt = \int_0^T a(t) h(t) dt$$

Physics-Informed Neural Networks (PINNs) I

As PINNs surgem como uma resposta à seguinte pergunta:

Pergunta Fundamental

Podemos treinar uma rede neural que **obedeça automaticamente às leis físicas** enquanto aprende com dados?

A resposta é sim — e elegante.

Physics-Informed Neural Networks (PINNs) II

O Problema Geral

Queremos resolver uma PDE:

$$\mathcal{N}[u](x, t) = 0$$

com condições:

$$u(x, 0) = u_0(x), \quad u(x, t)|_{\partial\Omega} = g(x, t)$$

Physics-Informed Neural Networks (PINNs) III

Ideia PINN

Definimos uma rede:

$$u_{\theta}(x, t)$$

que deve aproximar a solução verdadeira u .

A Loss total é:

$$\mathcal{L}(\theta) = \underbrace{\frac{1}{N_b} \sum_{i=1}^{N_b} \|u_{\theta}(x_i, t_i) - u_b^{(i)}\|^2}_{\mathcal{L}_{\text{boundary}}} + \lambda \underbrace{\frac{1}{N_r} \sum_{j=1}^{N_r} \|\mathcal{N}[u_{\theta}](x_j, t_j)\|^2}_{\mathcal{L}_{\text{física}}}$$

Physics-Informed Neural Networks (PINNs) IV

Autodiferenciação: A Chave das PINNs

Para cada ponto (x_j, t_j) :

$$\mathcal{N}[u_\theta](x_j, t_j)$$

é computado com derivadas obtidas via autograd.

Exemplo na equação do calor:

$$u_t = \alpha u_{xx}$$

PINN computa automaticamente:

$$u_t = \frac{\partial u_\theta}{\partial t}, \quad u_{xx} = \frac{\partial^2 u_\theta}{\partial x^2}$$

Physics-Informed Neural Networks (PINNs) V

Exemplo Completo: PINN para a Equação de Onda

$$u_{tt} = c^2 u_{xx}$$

Resíduo:

$$r(x, t) = u_{tt}(x, t) - c^2 u_{xx}(x, t)$$

Loss física:

$$\mathcal{L}_f = \frac{1}{N_r} \sum r(x_i, t_i)^2$$

O autograd calcula:

$$u_t, u_{tt}, u_x, u_{xx}$$

simplesmente aplicando a regra da cadeia.

Physics-Informed Neural Networks (PINNs) VI

Resumo

As PINNs substituem malhas numéricas complexas por:

sample points + autodiff

e funcionam como um método **mesh-free**.

GNNs = Discretização de Operadores Diferenciais I

Uma visão moderna: **GNNs** podem ser vistas como **PINNs** discretizadas em um grafo.

GNNs = Discretização de Operadores Diferenciais II

Operadores Diferenciais em Malhas

Considere a PDE elíptica:

$$-\Delta u = f$$

Num grafo com pesos w_{ij} , o Laplaciano discreto é:

$$(\Delta_G u)_i = \sum_{j \sim i} w_{ij} (u_j - u_i)$$

GNNs = Discretização de Operadores Diferenciais III

Interpretação

GNNs comuns (GCN, GraphSAGE, GAT) implementam **aproximações do Laplaciano**.

Logo... elas são **PINNs** onde a física = **regularização laplaciana**.

GNNs = Discretização de Operadores Diferenciais IV

GCN = Solver Poissoniano Discretizado

Uma camada GCN:

$$h^{(k+1)} = \sigma(\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} h^{(k)} W)$$

equivale a:

$$h^{(k+1)} = h^{(k)} + \text{Laplaciano Suavizado}$$

Isso lembra explicitamente um passo de Euler para a difusão:

$$\frac{dh}{dt} = -Lh$$

GNNs = Discretização de Operadores Diferenciais V

PINN em Grafo: Definição

Se desejarmos que a solução aprenda uma PDE na malha, podemos definir:

$$\mathcal{L}_f = \sum_i \|\mathcal{N}_G[h_\theta]_i\|^2$$

onde:

$$\mathcal{N}_G$$

é a versão discreta do operador diferencial.

GNNs = Discretização de Operadores Diferenciais VI

Conclusão

As GNNs são PINNs definidas em domínios discretos (malhas / grafos). Os Neural Operators generalizam isso para domínios contínuos.

Neural ODEs como RNNs de Passo Infinitesimal I

Uma RNN típica:

$$h_{t+1} = F(h_t, x_t)$$

Se o passo é pequeno e constante:

$$h(t + \Delta t) = h(t) + \Delta t f(h(t), x(t))$$

Neural ODEs como RNNs de Passo Infinitesimal II

Levando ao limite:

$$\frac{dh}{dt} = f(h(t), x(t))$$

Conclusão

RNN = Neural ODE com entradas discretas. Quando a entrada é contínua $\rightarrow$ ODE controlada.

Neural ODEs como RNNs de Passo Infinitesimal III

CDEs (Controlled Differential Equations)

É o formalismo moderno que substitui RNNs:

$$\frac{dh}{dt} = f(h(t)) \cdot \dot{x}(t)$$

onde $x(t)$ é o caminho da entrada.

Isso generaliza e contém:

- RNNs
- Neural ODEs
- ResNets

Neural ODEs como RNNs de Passo Infinitesimal IV

Processos Pontuais: Eventos Irregulares

Para modelar eventos (cliques, transações, spikes):

$$\lambda(t) = g(h(t))$$

$$\frac{dh}{dt} = f(h(t))$$

Eventos surgem de um processo Poisson condicional:

$$t_{k+1} \sim \text{Poisson}(\lambda(t))$$

Chamamos isso de:

- Neural Hawkes Process
- ODE-Hawkes
- Continuous-Time RNN

Resumo da Aula

- ResNets aproximam ODEs no limite.
- Neural ODE = forward com ODEsolve + backward com método adjunto.
- PINNs = redes que aprendem PDEs via autograd.
- GNNs são PINNs discretizadas (Laplaciano $\approx$ derivada segunda).
- Neural ODEs generalizam RNNs, Hawkes, e CDEs.

A matemática do tempo contínuo é a ponte entre redes profundas e sistemas dinâmicos.

Referências I